

Logistic Regression Part II

We have k classes. We should write equations for k classes like

$$Z_1 = W_{11}X_1 + W_{12}X_2 + W_{13}X_3 ++W_{1n}X_n+b_1$$

$$Z_2 = W_{21}X_1 + W_{22}X_2 + W_{23}X_3 ++ W_{2n}X_n +b_2$$

$$Z_3 = W_{31}X_1 + W_{32}X_2 + W_{33}X_3 + + W_{3n}X_n + b_3$$

$$.....$$

$$.....$$

$$Z_k = W_{k1}X_1 +W_{k2}X_2 + W_{k3}X_3++ W_{kn}X_n +b_k$$

X1	X2	X3		Xn	Class
...					A
.....					A
....					B
.....					C
....					
.....					
....					
.....					

We start with random values of all the Ws and bs. We are given a training dataset. We start with the 1st example. Its class is A. We calculate Z1 and find the probability of Class A. If it is correctly classified, we go to the next example. But if it is misclassified, we update the corresponding Ws and Bs using the formula $W_{ij} = W_{ij} + \eta(Y-\tilde{Y})X_i$

.....		
.....		

The multinomial logistic classifier uses a generalization of the sigmoid, called the **softmax** function, to compute $p(y_k = 1|\mathbf{x})$. The softmax function takes a vector $\mathbf{z} = [z_1, z_2, \dots, z_K]$ of K arbitrary values and maps them to a probability distribution, with each value in the range $[0,1]$, and all the values summing to 1. Like the sigmoid, it is an exponential function.

For a vector $\mathbf{z}$ of dimensionality K , the softmax is defined as:

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^K \exp(z_j)} \quad 1 \leq i \leq K \quad (5.16)$$

$$p(y_k = 1|\mathbf{x}) = \frac{\exp(\mathbf{w}_k \cdot \mathbf{x} + b_k)}{\sum_{j=1}^K \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)}$$

$$\text{softmax}(\mathbf{z}) = \left[\frac{\exp(z_1)}{\sum_{i=1}^K \exp(z_i)}, \frac{\exp(z_2)}{\sum_{i=1}^K \exp(z_i)}, \dots, \frac{\exp(z_K)}{\sum_{i=1}^K \exp(z_i)} \right]$$

Suppose $Z_1 = 0.6$, $Z_2 = 1.1$, $Z_3 = -1.5$, $Z_4 = 1.2$, $Z_5 = 3.2$ and $Z_6 = -1.1$

$$P(Y_1 = 1) = \frac{\text{Exp}(0.6)}{\text{Exp}(0.6) + \text{Exp}(1.1) + \text{Exp}(-1.5) + \text{Exp}(1.2) + \text{Exp}(3.2) + \text{Exp}(-1.1)}$$

$$P(Y_2 = 1) = \frac{\text{Exp}(1.1)}{\text{Exp}(0.6) + \text{Exp}(1.1) + \text{Exp}(-1.5) + \text{Exp}(1.2) + \text{Exp}(3.2) + \text{Exp}(-1.1)}$$

and so on.

$$P(Y_1, Y_2, Y_3, Y_4, Y_5, Y_6) = [0.05, 0.09, 0.01, 0.1, 0.74, 0.01]$$

Predicted Class is Y_5

Suppose the actual class is Y_2

$$\text{Loss} = -\log(0.09) = 2.41$$

$$W_{31} = W_{31} + 0.1 (1 - 0.09) X_1$$

$$W_{32} = W_{32} + 0.1 (1 - 0.09) X_2$$

For two class problem cross entropy loss $L_{CE}(\hat{y}, y) = -\log p(y|x) = -[y \log \hat{y} + (1-y) \log (1-\hat{y})]$

For more than two class problem and K number of examples the total entropy loss is

= $-\log Y_C - \log Y_A - \log Y_C - \log Y_B - \dots$ for K number of examples

$$L_{CE}(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{k=1}^K y_k \log \hat{y}_k$$

$$= -\log \hat{y}_c, \quad (\text{where } c \text{ is the correct class})$$

$$= -\log \hat{p}(y_c = 1|\mathbf{x}) \quad (\text{where } c \text{ is the correct class})$$

$$= -\log \frac{\exp(\mathbf{w}_c \cdot \mathbf{x} + b_c)}{\sum_{j=1}^K \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)} \quad (c \text{ is the correct class})$$

$$\frac{\partial L_{CE}}{\partial w_{k,i}} = -(y_k - \hat{y}_k)x_i$$

$$= -(y_k - p(y_k = 1|x))x_i$$

$$= -\left(y_k - \frac{\exp(\mathbf{w}_k \cdot \mathbf{x} + b_k)}{\sum_{j=1}^K \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)}\right)x_i$$

Stochastic vs Minibatch processing

$$\begin{aligned} \text{Cost}(\hat{y}, y) &= \frac{1}{m} \sum_{i=1}^m L_{\text{CE}}(\hat{y}^{(i)}, y^{(i)}) \\ &= -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log \sigma(\mathbf{w} \cdot \mathbf{x}^{(i)} + b) + (1 - y^{(i)}) \log (1 - \sigma(\mathbf{w} \cdot \mathbf{x}^{(i)} + b)) \end{aligned} \quad (5.33)$$

The mini-batch gradient is the average of the individual gradients from Eq. 5.30:

$$\frac{\partial \text{Cost}(\hat{y}, y)}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m [\sigma(\mathbf{w} \cdot \mathbf{x}^{(i)} + b) - y^{(i)}] x_j^{(i)} \quad (5.34)$$

Instead of using the sum notation, we can more efficiently compute the gradient in its matrix form, following the vectorization we saw on page 7, where we have a matrix $\mathbf{X}$ of size $[m \times f]$ representing the m inputs in the batch, and a vector $\mathbf{y}$ of size $[m \times 1]$ representing the correct outputs:

$$\begin{aligned} \frac{\partial \text{Cost}(\hat{y}, y)}{\partial \mathbf{w}} &= \frac{1}{m} (\hat{\mathbf{y}} - \mathbf{y})^\top \mathbf{X} \\ &= \frac{1}{m} (\sigma(\mathbf{X}\mathbf{w} + \mathbf{b}) - \mathbf{y})^\top \mathbf{X} \end{aligned} \quad (5.35)$$

1 Finding classes of m examples in binary classification

Suppose we have a data set having f number of attributes $x_1, x_2, \dots, x_f$ and we will find out the classes of m number of examples using the z score $z = w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_fx_f + b$ where all the w s and b are known. We will have a matrix having m rows and f columns to represent the m examples. We will have another matrix f rows and single column to represent the w s. If we multiply these two matrix having dimensions $m \times f$ and $f \times 1$, we will get an output matrix having dimension $m \times 1$. Every row represents the summation $w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_fx_f$ for each example. Now we add this $m \times 1$ matrix with $m \times 1$ column matrix (each element represents b) to get $m \times 1$ matrix representing the z values of each example. Applying σ function on each element thus gives the probability of belonging to $y = 1$ class for each of the m examples.

$$\begin{bmatrix} x_1^1 & x_2^1 & x_1^3 \dots & x_f^1 \\ x_1^2 & x_2^2 & x_3^2 \dots & x_f^2 \\ \dots & \dots & \dots & \dots \\ \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots \\ x_1^m & x_2^m & x_3^m \dots & x_f^m \end{bmatrix} \times \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ \vdots \\ w_f \end{bmatrix} = \begin{bmatrix} x_1^1 w_1 + x_2^1 w_2 + x_1^3 w_3 + \dots + x_f^1 w_f \\ x_1^2 w_1 + x_2^2 w_2 + x_3^2 w_3 + \dots + x_f^2 w_f \\ x_1^3 w_1 + x_2^3 w_2 + x_3^3 w_3 + \dots + x_f^3 w_f \\ \vdots \\ x_1^m w_1 + x_2^m w_2 + x_3^m w_3 + \dots + x_f^m w_f \end{bmatrix}$$
$$\sigma \left(\begin{bmatrix} x_1^1 w_1 + x_2^1 w_2 + x_1^3 w_3 + \dots + x_f^1 w_f \\ x_1^2 w_1 + x_2^2 w_2 + x_3^2 w_3 + \dots + x_f^2 w_f \\ x_1^3 w_1 + x_2^3 w_2 + x_3^3 w_3 + \dots + x_f^3 w_f \\ \vdots \\ x_1^m w_1 + x_2^m w_2 + x_3^m w_3 + \dots + x_f^m w_f \end{bmatrix} + \begin{bmatrix} b \\ b \\ b \\ \vdots \\ b \end{bmatrix} \right) = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \hat{y}_3 \\ \vdots \\ \hat{y}_m \end{bmatrix}$$

2 Finding classes of m examples in k -ary classification

Suppose we have a data set having f number of attributes $x_1, x_2, \dots, x_f$ and we will find out the classes of m number of examples where each example belongs to one of k classes. We have k number of z score $z_1, z_2, z_3, \dots, z_k$ for k different classes. Each z_i is expressed for j -th example by $z_i = w_{i1}x_1^j + w_{i2}x_2^j + w_{i3}x_3^j + \dots + w_{if}x_f^j + b_i$ where all the w_{ij} s and b_i are known. We will have a matrix having m rows and f columns to represent the m examples. We will have another matrix f rows and k columns to represent the w_{ij} s. If we multiply these two matrix having dimensions $m \times f$ and $f \times k$, we will get an output matrix having dimension $m \times k$. Every element represents the summation $w_{i1}x_1^j + w_{i2}x_2^j + w_{i3}x_3^j + \dots + w_{if}x_f^j$ for z_i score of each j -th example. Now we add this $m \times k$ matrix with $m \times k$ (each row represents b_i for $i = 1$ to $i = k$) to get $m \times k$ matrix where each row represents the z_i values of each i -th example. Applying function on each element of each row thus gives the probability of belonging to $y_k = 1$ class for each of the m examples.

$$\begin{aligned}
 & \begin{bmatrix} x_1^1 & x_2^1 & x_3^1 \dots & x_f^1 \\ x_1^2 & x_2^2 & x_3^2 \dots & x_f^2 \\ \dots & \dots & \dots & \dots \\ \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots \\ x_1^m & x_2^m & x_3^m \dots & x_f^m \end{bmatrix} \times \begin{bmatrix} w_{11} & w_{21} & w_{31} & \dots & w_{k1} \\ w_{12} & w_{22} & w_{32} & \dots & w_{k2} \\ w_{13} & w_{23} & w_{33} & \dots & w_{k3} \\ \vdots & \vdots & \vdots & \dots & \vdots \\ w_{1f} & w_{2f} & w_{3f} & \dots & w_{kf} \end{bmatrix} \\
 &= \begin{bmatrix} x_1^1 w_{11} + x_2^1 w_{12} + x_3^1 w_{13} + \dots + x_f^1 w_{1f} & x_1^1 w_{21} + x_2^1 w_{22} + x_3^1 w_{23} + \dots + x_f^1 w_{2f} & \dots & x_1^1 w_{k1} + x_2^1 w_{k2} + x_3^1 w_{k3} + \dots + x_f^1 w_{kf} \\ x_1^2 w_{11} + x_2^2 w_{12} + x_3^2 w_{13} + \dots + x_f^2 w_{1f} & x_1^2 w_{21} + x_2^2 w_{22} + x_3^2 w_{23} + \dots + x_f^2 w_{2f} & \dots & x_1^2 w_{k1} + x_2^2 w_{k2} + x_3^2 w_{k3} + \dots + x_f^2 w_{kf} \\ x_1^3 w_{11} + x_2^3 w_{12} + x_3^3 w_{13} + \dots + x_f^3 w_{1f} & x_1^3 w_{21} + x_2^3 w_{22} + x_3^3 w_{23} + \dots + x_f^3 w_{2f} & \dots & x_1^3 w_{k1} + x_2^3 w_{k2} + x_3^3 w_{k3} + \dots + x_f^3 w_{kf} \\ \vdots & \vdots & \dots & \vdots \\ x_1^m w_{11} + x_2^m w_{12} + x_3^m w_{13} + \dots + x_f^m w_{1f} & x_1^m w_{21} + x_2^m w_{22} + x_3^m w_{23} + \dots + x_f^m w_{2f} & \dots & x_1^m w_{k1} + x_2^m w_{k2} + x_3^m w_{k3} + \dots + x_f^m w_{kf} \end{bmatrix}
 \end{aligned}$$

$$\begin{bmatrix} x_1^1 w_{11} + x_2^1 w_{12} + x_3^1 w_{13} + \dots + x_f^1 w_{1f} & x_1^1 w_{21} + x_2^1 w_{22} + x_3^1 w_{23} + \dots + x_f^1 w_{2f} & \dots & x_1^1 w_{k1} + x_2^1 w_{k2} + x_3^1 w_{k3} + \dots + x_f^1 w_{kf} \\ x_1^2 w_{11} + x_2^2 w_{12} + x_3^2 w_{13} + \dots + x_f^2 w_{1f} & x_1^2 w_{21} + x_2^2 w_{22} + x_3^2 w_{23} + \dots + x_f^2 w_{2f} & \dots & x_1^2 w_{k1} + x_2^2 w_{k2} + x_3^2 w_{k3} + \dots + x_f^2 w_{kf} \\ x_1^3 w_{11} + x_2^3 w_{12} + x_3^3 w_{13} + \dots + x_f^3 w_{1f} & x_1^3 w_{21} + x_2^3 w_{22} + x_3^3 w_{23} + \dots + x_f^3 w_{2f} & \dots & x_1^3 w_{k1} + x_2^3 w_{k2} + x_3^3 w_{k3} + \dots + x_f^3 w_{kf} \\ \vdots & \vdots & \dots & \vdots \\ x_1^m w_{11} + x_2^m w_{12} + x_3^m w_{13} + \dots + x_f^m w_{1f} & x_1^m w_{21} + x_2^m w_{22} + x_3^m w_{23} + \dots + x_f^m w_{2f} & \dots & x_1^m w_{k1} + x_2^m w_{k2} + x_3^m w_{k3} + \dots + x_f^m w_{kf} \end{bmatrix}$$

$$+ \begin{bmatrix} b_1 & b_2 & b_3 & \dots & b_k \\ b_1 & b_2 & b_3 & \dots & b_k \\ b_1 & b_2 & b_3 & \dots & b_k \\ \vdots & \vdots & \vdots & \dots & \vdots \\ b_1 & b_2 & b_3 & \dots & b_k \end{bmatrix} = \begin{bmatrix} z_1^1 & z_2^1 & z_3^1 & \dots & z_k^1 \\ z_1^2 & z_2^2 & z_3^2 & \dots & z_k^2 \\ z_1^3 & z_2^3 & z_3^3 & \dots & z_k^3 \\ \vdots & \vdots & \vdots & \dots & \vdots \\ z_1^m & z_2^m & z_3^m & \dots & z_k^m \end{bmatrix}$$

$$\text{Softmax} \left(\begin{bmatrix} z_1^1 & z_2^1 & z_3^1 & \dots & z_k^1 \\ z_1^2 & z_2^2 & z_3^2 & \dots & z_k^2 \\ z_1^3 & z_2^3 & z_3^3 & \dots & z_k^3 \\ \vdots & \vdots & \vdots & \dots & \vdots \\ z_1^m & z_2^m & z_3^m & \dots & z_k^m \end{bmatrix} \right) = \begin{bmatrix} \hat{y}^1 \\ \hat{y}^2 \\ \hat{y}^3 \\ \vdots \\ \hat{y}^m \end{bmatrix}$$

Now to update weights for a batch of m training examples, we can use the following matrix algebra to update the weights in a single matrix multiplication

$$\begin{aligned}
\begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ \vdots \\ w_f \end{bmatrix} &= \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ \vdots \\ w_f \end{bmatrix} - \eta \times \frac{1}{m} \left(\begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \hat{y}_3 \\ \vdots \\ \hat{y}_m \end{bmatrix} - \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_m \end{bmatrix} \right)^T \times \begin{bmatrix} x_1^1 & x_2^1 & x_3^1 & \dots & x_f^1 \\ x_1^2 & x_2^2 & x_3^2 & \dots & x_f^2 \\ \dots & \dots & \dots & \dots & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ x_1^m & x_2^m & x_3^m & \dots & x_f^m \end{bmatrix} \\
&= \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ \vdots \\ w_f \end{bmatrix} - \eta \times \frac{1}{m} \left(\begin{bmatrix} \hat{y}_1 - y_1 \\ \hat{y}_2 - y_2 \\ \hat{y}_3 - y_3 \\ \vdots \\ \hat{y}_m - y_m \end{bmatrix} \right)^T \times \begin{bmatrix} x_1^1 & x_2^1 & x_3^1 & \dots & x_f^1 \\ x_1^2 & x_2^2 & x_3^2 & \dots & x_f^2 \\ \dots & \dots & \dots & \dots & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ x_1^m & x_2^m & x_3^m & \dots & x_f^m \end{bmatrix} \\
&= \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ \vdots \\ w_f \end{bmatrix} - \eta \times \frac{1}{m} \begin{bmatrix} (\hat{y}_1 - y_1)x_1^1 + (\hat{y}_2 - y_2)x_1^2 + (\hat{y}_3 - y_3)x_1^3 + \dots + (\hat{y}_m - y_m)x_1^m \\ (\hat{y}_1 - y_1)x_2^1 + (\hat{y}_2 - y_2)x_2^2 + (\hat{y}_3 - y_3)x_2^3 + \dots + (\hat{y}_m - y_m)x_2^m \\ (\hat{y}_1 - y_1)x_3^1 + (\hat{y}_2 - y_2)x_3^2 + (\hat{y}_3 - y_3)x_3^3 + \dots + (\hat{y}_m - y_m)x_3^m \\ \vdots \\ (\hat{y}_1 - y_1)x_f^1 + (\hat{y}_2 - y_2)x_f^2 + (\hat{y}_3 - y_3)x_f^3 + \dots + (\hat{y}_m - y_m)x_f^m \end{bmatrix}
\end{aligned}$$

